Sinkless Orientation in Incremental dynamic-LOCAL

Double-logarithmic deterministic and triple-logarithmic randomized locality

25 October 2026

Abstract

We maintain an orientation of an incrementally growing graph in which every vertex of degree at least three has an outgoing edge. We prove worst-case locality $O(\log \log n)$ deterministically and $O(\log \log \log n)$ with randomization against an oblivious insertion adversary. In the randomized result, for any fixed $c > 0$, one event of probability at least $1 - n^{-c}$ guarantees correctness after *every* update; the radius bound holds on every random tape.

The main argument is a problem-specific transfer from online to dynamic locality. First reduce monotonically to maximum degree three. Query a single online algorithm only at vertices whose surrounding neighborhoods have become permanent, and freeze their incident directions. Every remaining vertex is close to a vertex of degree at most two. Orienting a short path toward such a vertex repairs a sink without disturbing frozen outputs or creating another sink. An online radius r gives dynamic radius $2r + 4$, with no loss in success probability for an oblivious sequence.

For completeness, we also prove the static ingredients: the deterministic clustering-and-greedy algorithm and the randomized marking-and-shattering algorithm underlying the known sequential bounds. No algorithmic theorem from the cited literature is needed as a black box in the upper-bound proofs.

Contents

1 Problem, models, and statements **2**

2 A monotone reduction to maximum degree three **4**

3 The deterministic online-to-dynamic transfer **4**

4 Randomization: one static success event for all updates **7**

5 Static tools and the deterministic online algorithm **8**

6 The randomized online algorithm, with a full shattering proof **11**

7 Completing the upper bounds **16**

1 Problem, models, and statements

All input graphs are finite, simple, undirected, and have at most a known number $n \geq 2$ of vertices. Vertices have distinct identifiers, which are used only for deterministic choices. Graph distance is unweighted; the distance between distinct components is infinite. Write $B_G(v, t)$ for the closed radius-t ball. An orientation assigns one direction to each edge. A *defect* is a vertex of degree at least three and outdegree zero. A *sinkless orientation* is an orientation without defects. Thus vertices of degree zero, one, or two are exempt. This convention makes the problem well-defined throughout a graph's growth, including on forests.

1.1 Incremental dynamic locality

Initially the graph is empty. An update either inserts one edge between existing vertices or inserts one vertex together with any set of edges from it to existing vertices. Edges and vertices are never deleted, and parallel edges are not inserted. The algorithm must output an orientation after every update. Intermediate computations during an update are not outputs.

Global computation and persistent internal memory are unrestricted. Locality restricts changes to the *output*, not access to the input or running time. We use the endpoint encoding: a vertex reports the direction of every incident edge, and the reports at its two endpoints must agree. Reversing an old edge therefore changes both endpoints' outputs.

For precision, after insertion of uv our radius-R algorithms change old edge directions only if both endpoints of the changed edge lie in

$$B_G(u, R) \cap B_G(v, R), \tag{1}$$

where G is the graph *after* the insertion. The directions of newly inserted edges are also assigned in this region. For insertion of a vertex z, the corresponding allowed region is $B_G(z, R)$. These are the usual affected regions in the induced-neighborhood formulation of incremental dynamic-LOCAL [1]. In proofs it is convenient to first bound distance to the *nearer* endpoint. The elementary inclusion

$$B_G(u, a) \cup B_G(v, a) \;\subseteq\; B_G(u, a+1) \cap B_G(v, a+1) \tag{2}$$

converts that bound to (1).

Definition 1.1 (Randomized guarantee)**.** A randomized incremental algorithm has radius $R(n)$ and error $\varepsilon(n)$ against an *oblivious adversary* if:

1. its output changes obey the radius bound for every random tape; and
2. for every entire insertion sequence fixed independently of its private random bits, the probability that its outputs are legal after all updates is at least $1 - \varepsilon(n)$.

The sequence includes vertex identifiers and any ordering data supplied with an update. The algorithm receives updates one at a time and does not know the future sequence.

This is a Monte Carlo guarantee with a hard locality bound. It is stronger than a separate success guarantee for an individual update. Our randomized theorem does not assume that future updates can depend on observed random outputs. The deterministic theorem permits such adaptive choices.

Theorem 1.2 (Main results)**.** *Sinkless orientation on general graphs admits the following incremental* dynamic-LOCAL *algorithms:*

(i) a deterministic algorithm of worst-case locality $O(\log \log n)$;

(ii) for every fixed $c > 0$, a randomized algorithm of worst-case locality $O(\log \log \log n)$ and error at most n^{-c} in Definition 1.1.

Both statements allow edge insertions and vertex insertions with incident edges. Constants in the randomized locality may depend on c.

Iterated logarithms in asymptotic bounds are truncated below by one, so the statements include small values of n. Unless written ln, logarithms are base two. These bounds concern radius per update, not the number of changed edges, total computation, or amortized work.

1.2 The online intermediate model

In online-LOCAL, the input graph H is fixed but initially unknown. Vertices are queried one at a time in an arbitrary order. On query v, an algorithm of radius r sees the subgraph induced by the union of all radius-r balls of vertices queried so far, with their identifiers and any port labels. It has unrestricted persistent global memory. It must give v's final output, which can never be revised. For orientation, that output specifies every incident edge direction. At the end, all reports must agree and give a sinkless orientation. All algorithms below use $r \geq 1$; an extra layer can always be included when degree or boundary-incidence information is needed.

A randomized online algorithm has error ε if, for every fixed graph and fixed query order independent of its coins, its entire final output is legal with probability at least $1 - \varepsilon$. Its radius bound holds on every tape. These definitions follow the locality models of [1, 8]. In particular, information or provisional plans kept in private memory need not coincide with the final outputs already requested from vertices.

1.3 Proof plan and provenance

The dynamic argument is given first, in Sections 2–4. It transforms an online radius r into dynamic radius $2r + 4$. Then Sections 5–6 construct the required online algorithms and thereby complete Theorem 1.2.

The static bounds are established results. Ghaffari, Harris, and Kuhn proved the deterministic and randomized sequential bounds [3, Theorems 7.2 and 7.3]. Our deterministic construction follows the particularly simple greedy and clustering proof of Balliu et al. [2, Section 3]. The randomized construction follows the marking-and-shattering method of Ghaffari and Su [4] and its sequential adaptation in [3, Algorithm 3]. The composition of sequential phases goes back to Ghaffari, Kuhn, and Maus [5]; we give the easier global-memory online simulation explicitly. We include proofs of all of these ingredients, with the graph classes and error conventions needed here.

The additional ingredient is to delay irrevocable decisions until input neighborhoods have stabilized. This reverses, for this particular problem, the usual direction of simulation from dynamic to online locality [1]. Short local repair is related to the viewpoint of local mending [6]; here the algorithm maintains a particular family of repairable states rather than extending arbitrary partial solutions. Related repairs toward low-degree vertices or cycles appear in Nolin's recent simple distributed sinkless-orientation algorithm [10], whose complexity bounds concern distributed communication rounds.

Recent work of Bourreau et al. [7] obtains optimal logarithmic dynamic locality for bipartite 3-coloring and discusses the difficulty of using online decisions when the graph can still change. Sinkless orientation has a useful special feature: in a subcubic graph, a degree-three vertex is

saturated forever, while a nonsaturated vertex is exempt from the output constraint. Our proof uses both properties.

There is a relevant historical qualification. Earlier versions of [8] claimed a general dynamic derandomization theorem that would also imply the deterministic numerical bound here. That claim was withdrawn in the corrected version 4, Section 3.7 and the errata (13 February 2026). The proof below is independent of that claim. We make no general online-to-dynamic or dynamic derandomization assertion.

2 A monotone reduction to maximum degree three

Lemma 2.1. *An incremental sinkless-orientation algorithm on subcubic graphs yields one on general graphs with the same vertex bound, success probability, and locality, apart from a minimum radius of one. The reduction is deterministic and depends only on the insertion sequence.*

Proof. For each vertex, permanently select its first three arriving incident edges, or all of them if fewer have arrived. In a simultaneous arrival, use a fixed identifier order. Let H contain exactly those edges selected by both endpoints. The graph H has maximum degree three and only grows: the status of an edge is decided when it arrives and never changes.

Run the subcubic algorithm on H. Orient an edge selected by exactly one endpoint out of that endpoint, permanently. Orient an edge selected by neither endpoint according to any fixed rule. Use the algorithm's directions for the edges of H.

If $\deg_G(v) \geq 3$, then v has selected three edges. If one of them was selected only by v, it is an outgoing edge. Otherwise all three belong to H, so $\deg_H(v) = 3$ and the orientation of H gives v an outgoing edge. Thus correctness in H implies correctness in G at every time.

An original edge insertion either inserts that same edge into H or leaves H unchanged. Since $H \subseteq G$, distances in G are no larger than distances in H, so every permitted change in H is permitted in G. All other old edges keep their directions. There are no new vertices. The entire support-graph sequence is fixed when the original insertion sequence is fixed, so the probability guarantee is preserved as well. □

We prove the subcubic algorithm for single-edge insertions on a fixed set of n initially isolated virtual vertices. Actual vertex arrivals can be mapped to unused virtual identifiers in $[n]$ in arrival order; physical identifiers need not be used internally. If ports are needed, assign them permanently in support-edge arrival order. Unused vertices remain isolated and impose no constraint. Section 3.4 explains why inserting a vertex with several edges preserves the same locality bound.

3 The deterministic online-to-dynamic transfer

Theorem 3.1. *Suppose A is a deterministic* online-LOCAL *algorithm for sinkless orientation on subcubic graphs with n vertices and radius $r = r(n) \geq 1$. There is a deterministic incremental* dynamic-LOCAL *algorithm on the same graph class with radius at most $2r + 4$.*

3.1 Free and committed vertices

Set $\tau = r + 1$. In the current subcubic graph H, define

$$F = \{v : \deg_H(v) \leq 2\}, \qquad \delta(v) = \operatorname{dist}_H(v, F), \qquad C = \{v : \delta(v) > \tau\}, \qquad U = V(H) \setminus C. \quad (3)$$

If a component has no vertex in F, its distances to F are infinite. Vertices in F are *free*, and vertices in C are *committed*. We run one persistent instance of A, querying a vertex exactly when it first joins C. Its incident edge directions are then frozen. Vertices in U have temporary directions, which are never passed to A as if they were outputs of A.

Lemma 3.2 (Monotonicity)**.** *Under subcubic edge insertions, $\delta(v)$ never decreases. Hence C only grows, and each vertex is queried at most once.*

Proof. Immediately before inserting ab, both a and b have degree at most two, so both are in the old free set. The new free set is a subset of the old one; only endpoints whose degrees reach three can leave it.

Consider a shortest new path from x to a new free vertex. If it does not use ab, it was already an old path to an old free vertex. If it uses ab, its prefix up to the first endpoint of ab is an old path to an old free vertex. In either case the old distance to the free set is at most the length of the new path. If the new distance is infinite there is nothing to prove. □

Lemma 3.3 (Permanence of revealed input)**.** *When x enters C, its radius-r online view, including one additional boundary layer, is permanent. At every later time the complete transcript of queries to A is a valid online prefix on the current graph, and also on every future graph in the insertion sequence.*

Proof. At commitment, every vertex within distance $r+1$ of x has degree three. No edge can subsequently be inserted incident to any of those vertices. The old radius-r ball can therefore neither lose an edge nor acquire a new incident edge.

Every later path from x of length at most $r+1$ uses only edges present at commitment time. Otherwise its first new edge has an old-edge prefix of length at most r, so its first endpoint was already saturated. That edge could not have been inserted. Consequently no new shortcut changes either ball or its boundary data.

Finally, all vertices in all previously revealed radius-r balls are saturated. Consequently no subsequent insertion changes an edge induced by their union or their incidences. Edges revealed when a further ball is added are consistent with the same current or future graph. The identifiers, ports, graph views, and query order are therefore exactly those of an online prefix on that fixed graph. This proves the assertion for the union-of-balls model, not only for separately viewed balls. □

In particular, the online outputs at committed vertices are consistent and give each of them an outgoing edge. To see this formally, extend the query prefix by all remaining vertices of the current graph. Correctness of A on that full online execution implies correctness and agreement of all the irrevocable outputs already produced. No future input is needed to run the simulation.

Lemma 3.4 (New commitments are nearby)**.** *After insertion of ab, every newly committed vertex is within distance τ of a or b in the updated graph.*

Proof. A newly committed vertex x had an old path of length at most τ to an old free vertex f. If f were still free, the same path would contradict the new condition $\delta(x) > \tau$. Thus f has ceased to be free, which can happen only to a or b. The old path is still present. □

3.2 Repair without touching committed edges

Lemma 3.5 (Full-path repair)**.** *Let the orientation be legal at every vertex of C. Every defect $x \in U$ can be removed by changing directions only on a path of length at most τ contained in U. This operation creates no new defect and changes no edge incident to C.*

Proof. Choose a shortest path

$$x = x_0, x_1, \ldots, x_k = f \in F, \qquad k \le \tau.$$

Every vertex x_i on it has distance at most $k - i \le \tau$ to F, and thus belongs to U. Orient the entire path as

$$x_0 \longrightarrow x_1 \longrightarrow \cdots \longrightarrow x_k.$$

The initial vertex and every internal vertex now have an outgoing path edge. The last vertex has degree at most two and is exempt. Vertices off the path have no incident direction changed. Hence no new defect appears, and the original one disappears. Both endpoints of every changed edge are in U. □

This argument does not require the path to be directed before repair. It does not require spare outgoing edges at intermediate vertices either: the new path itself supplies their outgoing edges.

3.3 The update rule and its radius

Maintain the following invariant: the orientation is legal, the vertices already queried from A are exactly C, and every edge incident to C has the direction specified by A. Initially C is empty and there are no edges, so the invariant holds.

On insertion of ab, perform these steps:

1. Give ab a provisional direction and recompute F, C, U for the updated graph. Let W be the entire set of vertices newly entering C.
2. Query the single maintained instance of A at all vertices of W, in identifier order. Set their incident directions to the returned values. Previously committed reports and the new reports agree by Lemma 3.3.
3. While a defect remains in U, apply Lemma 3.5, using fixed tie-breaking rules to choose a defect and a shortest path.

All of W is computed before any repair. If several nearby vertices lose their last short route to a free vertex, *all* of them are committed in step 2. Commitment changes orientations, not the degree-based set F.

Proof of Theorem 3.1. Step 2 makes every vertex of the new C legal and preserves the old committed directions. Step 3 preserves them and removes at least one defect without creating another. It terminates and restores the invariant.

It remains to bound all changes during the update. An endpoint of an edge changed in step 2 is either newly committed or adjacent to a newly committed vertex. By Lemma 3.4, it is within distance $\tau + 1$ of a or b. Before step 2, only a or b can have become defective because of the insertion. After step 2, every defect is therefore within distance $\tau + 1$ of an insertion endpoint.

Every later repair starts at one of these same defects: no repair creates a new one. Its path has length at most τ. Thus every endpoint of every changed edge is within distance

$$(\tau + 1) + \tau = 2\tau + 1 = 2r + 3$$

of a or b. The repairs do not accumulate radius even when there are many defects. Inclusion (2) gives the claimed radius $2r + 4$ in the precise convention (1). □

The proof includes an update that closes an entire component into a cubic graph. Then that component has no free vertex and all its uncommitted vertices join C. Lemma 3.4 still places every one of them near the insertion. No separate closing-component procedure is needed.

3.4 Vertex insertions

To insert a vertex z with its incident edges, first activate its previously isolated virtual vertex, and then insert its new edges one at a time in identifier order. Do not expose the intermediate outputs. For a substep inserting zv, the allowed region (1) is contained in $B(z, R)$ in the substep graph and hence also in the final graph of the update. The union of all substep changes is therefore within $B(z, R)$. The same refinement applies to Lemma 2.1. Thus vertex insertions, even with many incident edges in the original graph, do not increase the radius. An isolated vertex arrival changes no old direction.

4 Randomization: one static success event for all updates

Theorem 4.1 (Probability-preserving transfer)**.** *Suppose a randomized* online-LOCAL *algorithm on subcubic graphs has hard radius $r(n) \geq 1$ and, for every fixed input graph and fixed query order, global failure probability at most $\varepsilon(n)$. Then there is a randomized incremental* dynamic-LOCAL *algorithm with hard radius $2r(n) + 4$ and error $\varepsilon(n)$ against an oblivious insertion adversary.*

Proof. Use the deterministic construction with the randomized algorithm A and one persistent random tape. The sets F, C, U, the order within each newly committed batch, and the refinement of vertex updates all depend only on the input history. They do not depend on orientations or on random bits.

First specify behavior even on tapes on which A fails. In step 2 collect all proposed outputs before changing any old direction. Check that they are well-formed, agree at the two endpoints whenever both are committed, agree with all old committed directions, and give every newly committed vertex an outgoing edge. If a check fails, reject the batch, set a permanent failure flag, and make no old-edge change. From then on, keep all old directions and orient new edges by a fixed rule. These exceptional executions need not remain legal, but they always obey the radius bound. If validation succeeds, the same repair and locality proof as before applies. This makes the algorithm a total Monte Carlo algorithm with a hard radius on every tape.

Now fix an entire insertion sequence. Let H_* be its final graph on the n virtual vertices. Let σ list, in their scheduled order, every vertex that ever enters C, followed by all other vertices in identifier order. Both H_* and this *planned* schedule are independent of the random tape. They are used for the analysis only.

Consider one full execution of A on (H_*, σ), using the dynamic simulation's tape. By Lemma 3.3, each input view supplied by the dynamic simulation is exactly the corresponding view in that fixed execution. Induction over calls to A shows that the transcripts and outputs coincide up to any rejected batch. The hypothetical schedule is defined whether or not the actual simulation aborts.

Let E be the event that this one complete static execution is legal. Then $\mathbb{P}(E) \geq 1 - \varepsilon(n)$. On E, every proposed batch is legal and compatible with every earlier batch, so no rejection occurs. At every update, the committed vertices are legal and Lemma 3.5 repairs the uncommitted region. All dynamic outputs are therefore legal on E. This proves the whole-execution probability guarantee with no union bound over updates. □

Remark 4.2 (Scope of the adversary assumption)**.** If future insertions depend on observed randomized outputs, H_* and σ can depend on the tape, so the fixed-input success theorem for A does not justify the last argument. Even fixing the final graph while allowing a coin-dependent insertion order is a different adversary model. The theorem proved here is precisely the oblivious-sequence statement of Definition 1.1.

5 Static tools and the deterministic online algorithm

We now prove the online bounds used by the transfer. Intermediate graphs may be loopless multigraphs: parallel edges are distinct edges, count separately toward degree, and can form a cycle of length two. This causes no problem for orientation. Identifiers need only be distinct; no bound on their numerical range is used.

5.1 Composing phases without looking farther than necessary

It is convenient to describe a construction as several online phases. Each phase produces an immutable label at each queried vertex, and a later phase uses earlier phases' labels as additional input. Global private memory is permitted in every phase. The following elementary simulation includes the independence property needed in the randomized proof.

Lemma 5.1 (Fixed-cone composition)**.** *On a fixed base graph, suppose there are p online phases with predetermined radii $\rho_1, \dots, \rho_p$. A phase can inspect the earlier phases' labels in its revealed neighborhoods. The final labels can be computed by one online algorithm of radius $\sum_i \rho_i$. Moreover, for a fixed graph and fixed external query order, each phase's query order can be chosen independently of all random bits, even when some vertices are irrelevant to that phase's task.*

Proof. Set $s_i = \sum_{j>i} \rho_j$, so $s_p = 0$. On an external query v, process phases $i = 1, \dots, p$ in order. In phase i, query every as yet unqueried vertex of $B(v, s_i)$ in identifier order, storing its returned label. Irrelevant vertices receive dummy labels but are still included in this schedule.

If phase i is queried at $u \in B(v, s_i)$, every needed earlier-phase label lies in $B(v, s_i + \rho_i)$. For each $j < i$, $s_j \geq s_i + \rho_i$, so phase j has already been processed at all those vertices. The input required to make this call is within distance $s_i + \rho_i \leq \sum_j \rho_j$ of v. The same statement holds for the unions of previously revealed balls, whose data have been cached.

To verify the simulation, list the calls made to each phase in their actual order. Imagine first executing phase 1 on its entire list, then phase 2 on its entire list, and so on. A call sees exactly the same earlier-phase labels as in the simulation, since all labels it inspects had already been computed and never change. Its same-phase history and random tape are also identical. Induction over phases and calls proves equality of the outputs. After all external vertices are queried, every phase's list includes all vertices. Finally, the lists were constructed from balls of the fixed base graph and identifier orders only; they do not depend on label values or coins. □

This is the online, global-memory version of sequential composition [5, 2]. In particular, the lemma never uses balls in a coin-dependent residual graph to decide which earlier preprocessing calls to make.

5.2 A greedy orientation for high-degree vertices

Lemma 5.2 (High-degree greedy algorithm)**.** *Let Q be a loopless multigraph each of whose connected components has at most $M \geq 2$ vertices, and put $h = \lfloor \log_2 M \rfloor + 1$. There is a greedy edge-by-edge orientation, correct for every edge order, which gives every vertex of degree at least h an outgoing edge. To process an edge, it needs only its endpoints' degrees, counts of processed incident edges, and whether they already have outgoing edges.*

Proof. Call a vertex *satisfied* if its degree is less than h or an edge has already been oriented out of it. Process edges once, in any order.

1. If at least one endpoint is satisfied, orient the edge toward a satisfied endpoint, with deterministic tie-breaking.

2. Otherwise orient it toward the endpoint with fewer already processed incident edges, again breaking ties deterministically.

Satisfaction is permanent. Consider the auxiliary graph consisting only of edges processed by rule 2. Maintain two invariants: each of its components has at most one unsatisfied vertex, and a component containing an unsatisfied vertex with b incoming edges has at least 2^b vertices.

Initially the assertions hold. Rule 1 cannot give another incoming edge to an unsatisfied vertex: if one endpoint is unsatisfied, the edge points *out* of it. Thus this rule can only remove unsatisfied vertices and does not damage the invariants.

In rule 2, the two unsatisfied endpoints belong to distinct auxiliary components by the first invariant. Suppose their incoming counts are $b_u \geq b_v$, and the edge is oriented $u \to v$. Since neither endpoint has an outgoing edge, these counts equal their processed-edge counts. The two components merge, u becomes satisfied, and v is the only possibly unsatisfied vertex of the merged component. Its new incoming count is $b_v + 1$, and the component has at least

$$2^{b_u} + 2^{b_v} \geq 2^{b_v+1}$$

vertices. This establishes both invariants.

If a degree-at-least-h vertex were still unsatisfied at the end, all of its edges would enter it, so its auxiliary component would contain at least $2^h > M$ vertices. It lies within one connected component of Q, a contradiction. □

The same guarantee applies as soon as all edges incident to a particular vertex have been processed. Complete the remaining edge order hypothetically: those incident directions cannot change, and the full execution is correct by Lemma 5.2.

5.3 Clusters that either have many boundary edges or can absorb a sink

Fix an integer $t \geq 1$ and set $L = 2t + 1$. Greedily construct a maximal set I of centers with pairwise distance greater than L: on processing a vertex, select it if no previously selected center is within distance L. This is an online phase of radius L. After all center decisions are made, assign a vertex x to the center minimizing

$$(\mathrm{dist}(x, a), \mathrm{ID}(a)) \quad \text{over } a \in I$$

in lexicographic order. Write C_a for the resulting cluster.

Lemma 5.3. *Every cluster C_a is connected, contains $B(a, t)$, and is contained in $B(a, L)$. If a cluster contains no cycle and no vertex of original degree at most two, then its number of boundary edges is at least 2^t.*

Proof. Maximality of I implies that every vertex is within distance L of a center, proving the radius bound. If $\mathrm{dist}(x, a) \leq t$, every other center b has $\mathrm{dist}(x, b) \geq \mathrm{dist}(a, b) - \mathrm{dist}(a, x) > 2t + 1 - t > t$, so x belongs to C_a.

For connectedness, let x choose a, and let y be the next vertex on a shortest x–a path. If some center b had a better distance–identifier pair at y, the triangle inequality would give a better pair for b at x too. Thus y also chooses a, and repeating the argument gives a path inside C_a from x to a.

Now suppose C_a is acyclic and all its vertices have original degree at least three. Its induced subgraph is a tree. Its contained radius-t ball has at least $3 \cdot 2^{t-1}$ vertices at distance t from a:

the root has at least three children, and every vertex at a smaller positive depth has at least two children. A missing or identified child would contradict either full degree in that ball or acyclicity. In particular $|C_a| \geq 2^t$. If $b(C_a)$ counts its boundary edges with multiplicity, the degree sum identity gives

$$b(C_a) = \sum_{x \in C_a} \deg_G(x) - 2(|C_a| - 1) \geq |C_a| + 2 \geq 2^t.$$

□

Call a cluster *absorbing* if it contains a degree-at-most-two vertex or a cycle. Its internal edges can be oriented so that all its constrained vertices have outgoing edges regardless of boundary directions. For a free vertex, orient a spanning tree toward it. For a cycle, orient that cycle cyclically and orient a forest spanning the remaining vertices toward the cycle. Orient unused edges arbitrarily. All choices use identifiers and edge identities, so every vertex can reconstruct the same orientation.

More generally, if a cluster has an outgoing boundary edge at w, orient a spanning tree toward w. Then w is satisfied by that boundary edge and all other cluster vertices are satisfied by the tree. These observations apply to a two-edge cycle in a multigraph as well.

5.4 The deterministic construction and its online implementation

Theorem 5.4 (Componentwise deterministic static bound)**.** *Let G be a loopless multigraph each of whose connected components has at most $M \geq 2$ vertices. Sinkless orientation can be computed by a deterministic online algorithm of radius $O(\log \log M)$. It needs only the bound M, not component sizes or the number of components. If a component exceeds M, the same algorithm still returns a consistent orientation within the same radius, although that component may contain defects.*

Proof. Put

$$h = \lfloor \log_2 M \rfloor + 1, \qquad t = \lceil \log_2 h \rceil, \qquad L = 2t + 1.$$

Construct the clusters above. Contract each cluster to a vertex, keeping every intercluster edge as a distinct edge of a loopless quotient multigraph Q. Each component of Q has at most M vertices. Apply Lemma 5.2 to its edges.

A quotient vertex of degree at least h has an outgoing edge. A cluster with fewer than h boundary edges must be absorbing by Lemma 5.3, since $2^t \geq h$. Thus every cluster either has an outgoing boundary edge or is absorbing. Orient its interior as described above. The resulting orientation is sinkless.

Here is a complete local online implementation, avoiding any assumption that the full quotient graph is already known. Use three phases on G.

1. Compute the greedy center set with radius $\rho_1 = L$.

2. At each vertex v, compute and store the full description of its cluster, including its induced edges, original vertex degrees, boundary edges, and the owners of their other endpoints. Radius $\rho_2 = 3L + 3$ in the graph labeled with the completed center decisions suffices. Indeed, v's center is within L, its entire cluster is within $2L$ of v, and owners of boundary neighbors can be determined from centers within a further $L + 1$ hops. The spare layers provide every needed incidence and degree.

3. On the first query at a member v of a cluster C, process every still unoriented boundary edge of C by the high-degree greedy rule, in fixed order. Store the directions globally. For each adjacent cluster, its complete phase-2 description is available at the other endpoint of the boundary edge, which lies within $2L + 1$ of v. Degrees, counts of previously processed edges,

and outgoing-edge status of quotient vertices can therefore be obtained from these descriptions and the persistent greedy history. Then orient and store the entire interior of C. Return the directions at v. On a later query in C, return the stored directions. This phase has radius $\rho_3 = 2L + 2$.

Processing a cluster finishes all of its boundary edges, so neither their directions nor its chosen interior ever changes afterward. Each quotient edge is processed exactly once. The resulting edge order can be arbitrary; Lemma 5.2 applies to all orders. Returning all incident directions therefore gives consistent immutable endpoint outputs.

By Lemma 5.1, the three phases require radius at most $6L + 5 = O(\log \log M)$ in the original graph. The argument uses the bound M only in the high-degree lemma, component by component. If the bound is false, make the algorithm total as follows: whenever a cluster has neither an outgoing boundary edge nor an absorbing structure, orient its spanning tree toward its least-identifier vertex and orient the rest arbitrarily. This may leave one defect, but produces a consistent orientation by the same local computations. Thus the radius bound is unconditional. □

In particular, taking $M = n$ gives the deterministic online bound required by Theorem 3.1. The componentwise formulation will also let us solve many small randomized residual components without exploring an entire component first.

6 The randomized online algorithm, with a full shattering proof

Theorem 6.1 (Randomized static bound)**.** *For every fixed $c > 0$, sinkless orientation on a subcubic graph with a known vertex bound $n \geq 2$ has a randomized online algorithm of radius $O_c(\log \log \log n)$ and global error at most n^{-c} for every fixed graph and fixed query order. The radius holds on every random tape.*

The proof has three combinatorial steps: replace constant-radius clusters by vertices with many available incidences, randomly satisfy most of those vertices, and use Theorem 5.4 on the remaining small components. We then explain the complete online implementation. This is the marking approach of [4, 3], with the reductions and probability estimate given here explicitly.

6.1 Constant-radius clusters and half-edges

Fix the absolute constants

$$a = 8, \qquad D = 512.$$

Greedily choose a maximal set of centers at pairwise distance greater than $2a = 16$, and assign every vertex to its nearest center, breaking ties by center identifier. The proof of Lemma 5.3 applies with this separation too: clusters are connected, have radius at most 16 and strong diameter at most 32, and contain the full radius-8 ball of their center. Indeed, a different center is at distance at least $17 - 8 = 9$ from a vertex within distance 8 of the chosen center.

As before, a cluster is absorbing if it contains a cycle or a vertex of degree at most two. Fix an internal orientation satisfying all its degree-three vertices, independent of its boundary. Call every other cluster *active*. An active cluster C is a tree whose vertices all have original degree three. Its radius-8 ball is a full cubic tree, and therefore

$$|C| \geq 1 + 3\sum_{i=0}^{7} 2^i = 766, \qquad b(C) = 3|C| - 2(|C| - 1) = |C| + 2 \geq 768 > D. \tag{4}$$

At each active cluster select exactly D boundary edges by a fixed order on their original endpoint identifiers. Form a quotient instance Q with one vertex per active cluster. An original boundary edge selected at both endpoints becomes an ordinary quotient edge. If selected at exactly one endpoint, it becomes a *half-edge* at that endpoint. If selected at neither endpoint, discard it from Q and fix an arbitrary direction for it. An absorbing endpoint never selects an edge.

A half-edge has one represented endpoint, and can be directed inward or outward relative to it. An outward half-edge satisfies that endpoint. Each item of Q, ordinary edge or half-edge, corresponds to a distinct original boundary edge. Thus any orientation of all the items maps consistently back to original edges. Every quotient vertex has exactly D incident items, counted with multiplicity. The ordinary underlying graph has maximum degree at most D; parallel edges are permitted, and there are no loops.

If every active quotient vertex has an outgoing item, every active original cluster has an outgoing boundary edge. Orient its internal tree toward the tail of one such edge. Absorbing clusters use their fixed internal solutions. This gives a sinkless orientation of the original graph. An unrepresented endpoint of a half-edge causes no conflict: it must obtain its own witness from its own selected items, or it belongs to an absorbing cluster. It places no requirement on that particular edge.

All of this preprocessing has constant radius. A step in the quotient corresponds to travel within a cluster and across one original boundary edge, costing at most $32 + 1 = 33$ original hops, with an additive constant to inspect endpoint clusters. We will ensure below that the entire preprocessing is independent of the later random bits.

6.2 Marking and a residual orientation problem

Independently for every item of Q, mark it with probability $1/4$. Give each marked item a uniformly random direction. There is one common random choice per ordinary edge, not separate choices at its endpoints; for a half-edge the choices are inward and outward.

Let X_v count the marked items incident to a quotient vertex v, and let Z_v be the event that v has no outgoing marked item. Classify bad vertices in the following priority order:

1. *Type I:* $X_v > D/2$.
2. *Type II:* not type I, but adjacent by an ordinary edge to a type I vertex.
3. *Type III:* neither type I nor type II, and Z_v occurs.

All remaining vertices are good. Let B be the set of bad vertices. Unmark every item incident to a type I vertex, and freeze the directions of all marks that remain. For a half-edge, only its represented endpoint counts as incident; the other original cluster has no role in this rule.

Lemma 6.2. *Every good vertex has an outgoing marked item that remains frozen. Every bad vertex has at least $D/2$ unmarked incident items.*

Proof. A good vertex has an outgoing marked item because it is not type III, and no such item is unmarked: neither the vertex nor any ordinary neighbor is type I. A type I vertex has all its D items unmarked. Any other bad vertex initially had at most $D/2$ marks, and unmarking only increases the number of unmarked items. □

Keep unmarked edges between bad vertices as residual ordinary edges, and replace an unmarked edge between a bad and a good vertex by a residual half-edge at the bad vertex. An unmarked original half-edge at a bad vertex remains a half-edge. Orient all other unmarked items arbitrarily. The residual instance has vertex set B, with at least $D/2 = 256$ incident items at each vertex. A

sinkless orientation of this instance finishes the orientation of Q: every bad vertex gets an outgoing item, while no good vertex loses its frozen outgoing witness.

6.3 Why every residual component is small with high probability

We prove the probability bound instead of invoking a shattering theorem. All distances and neighborhoods in this subsection use the ordinary underlying graph of Q and ignore half-edges.

Define a *primitive rare event*

$$P_v = \{X_v > D/2\} \cup Z_v.$$

It depends only on the independent random choices for items incident to v. Consequently these events are mutually independent at any set of pairwise nonadjacent vertices. This remains true for parallel edges and half-edges. We use primitive events rather than literal type III events, since the latter also inspect neighbors' type I status.

An exponential-moment estimate and independence of the D incident items give

$$\mathbb{P}[X_v > D/2] \le 2^{-D/2}\mathbb{E}[2^{X_v}] = 2^{-D/2}(5/4)^D \le e^{-D/12}, \tag{5}$$

$$\mathbb{P}[Z_v] = (7/8)^D \le e^{-D/8}. \tag{6}$$

For the last inequality in (5), it suffices to use $\ln(5/4) \le 1/4$ and $\ln 2 \ge 2/3$. The latter follows, for example, by integrating the convex function $1/x$ on $[1, 2]$ and using its midpoint value. Thus

$$\mathbb{P}[P_v] \le p := 2e^{-D/12}. \tag{7}$$

Let P be the set of vertices where P_v occurs, and let I be the type I set. The definitions imply

$$B = P \cup N_Q(I). \tag{8}$$

In particular, each bad vertex is primitive or adjacent to a primitive vertex, and such a witnessing neighbor lies in the same bad component.

Lemma 6.3 (Separated witnesses)**.** *If a component of $Q[B]$ has at least $2D^2k$ vertices, there is a set of k primitive vertices that are pairwise nonadjacent in Q and connected by pairs whose Q-distance is between two and four.*

Proof. Choose any primitive vertex in the component, which exists by (8), and start a set S with it. A radius-two ball in a graph of maximum degree at most D has at most $1+D+D(D-1) = D^2+1 \le 2D^2$ vertices. While $|S| < k$, the radius-two neighborhood of S therefore does not cover the component.

Take a path inside the component from S to an uncovered vertex. Distances to S change by at most one along an edge, so this path contains a vertex w at Q-distance exactly three from S. If w is primitive, add it to S. Otherwise, by (8), it has a type I neighbor z in the same component. Add z instead. In the latter case $2 \le \mathrm{dist}_Q(S, z) \le 4$; in the former case the distance is three. The new vertex is nonadjacent to every previous choice and within distance four of at least one. Induction gives the required independent, connected witness set. □

Let J connect pairs of vertices at Q-distance two, three, or four. Its maximum degree is at most

$$D^2 + D^3 + D^4 \le 2D^4 =: \Lambda.$$

There are at most $n(4\Lambda)^{k-1}$ rooted ordered trees with k distinct vertices embedded in J. Indeed, at most 4^{k-1} tree shapes are possible: a depth-first traversal is encoded by $k-1$ opening and $k-1$

closing steps. There are at most n choices for the root and at most Λ choices for the image of each child. Restricting to injective embeddings whose images are independent in Q only decreases this count.

For any such independent image, all k primitive events occur with probability at most p^k. Every witness from Lemma 6.3 has a spanning tree counted above. A union bound therefore gives

$$\mathbb{P}[\text{a primitive witness of size } k \text{ exists}] \le n(4\Lambda)^{k-1}p^k \le n(4\Lambda p)^k. \tag{9}$$

The fixed choice $D = 512 = 2^9$ satisfies

$$4\Lambda p \le 16D^4 e^{-D/12} = \exp(40\ln 2 - 128/3) < e^{-2}.$$

For the chosen confidence exponent $c > 0$, set

$$k = \lceil (c+1)\ln n \rceil, \qquad m_0 = 2D^2 k. \tag{10}$$

Equations (9) and (10) show that the probability of a bad component with at least m_0 vertices is at most

$$ne^{-2k} \le n^{-2c-1} \le n^{-c}. \tag{11}$$

The residual ordinary graph only deletes edges from $Q[B]$. Thus with probability at least $1 - n^{-c}$, *all* its components have fewer than $m_0 = O_c(\log n)$ vertices. Call this the shattering event.

6.4 Cleaning up residual half-edges

Replace each residual half-edge by an ordinary edge to its own fresh degree-one leaf. The resulting graph R is a loopless multigraph. Old residual vertices have degree at least 256, while the leaves are exempt from the sinkless requirement. A direction on a leaf edge translates directly to a direction on its half-edge.

There are at most D such leaves per residual vertex. On the shattering event, every component of R consequently has at most

$$M = (D+1)m_0 = O_c(\log n) \tag{12}$$

vertices. Give all virtual vertices distinct tuple identifiers, derived from their cluster identifiers and, for leaves, their original edge identities, with separate type tags for quotient vertices and leaves. Apply Theorem 5.4 to R with the numerical parameter M. It gives each old residual vertex an outgoing edge, using the explicit radius from that theorem's proof,

$$q = 12\left\lceil \log_2\left(\lfloor \log_2 M \rfloor + 1\right)\right\rceil + 11 = O_c(\log\log\log n). \tag{13}$$

No routine first explores or counts a residual component. The algorithm uses this same bound M in every component, regardless of the total number of virtual vertices or the range of their identifiers. On tapes where a component exceeds M, its off-promise behavior still uses radius q and returns consistent directions, by Theorem 5.4.

6.5 A fully local online simulation

It remains to justify both online locality and the independence assumed in the probability calculation. In particular, an online algorithm cannot silently assume that all clusters were computed globally in advance.

Use Lemma 5.1 on the original graph G with the following six phases. Each phase returns an immutable label at *every* original vertex, using a dummy label when its particular task is irrelevant there. The numerical radii below are deliberately loose constants.

1. *Centers*, radius 16: compute the greedy set of centers at distance greater than 16.

2. *Owners*, radius 16: read the completed center labels and choose the nearest center, breaking ties by identifier.

3. *Cluster descriptions*, radius 100: store at each member its cluster's entire induced graph, degrees, boundary edges, absorbing status, and selected D incidences if active. A member is within distance 32 of every vertex in its cluster, and its boundary neighbors are within distance 33, so the stated radius more than suffices. Store a canonical internal solution for each absorbing cluster.

4. *Random marking*, radius 100: construct the selected quotient incidences, generate their marks and directions, and record types and surviving marks. To inspect an ordinary neighbor's type I event it suffices to inspect that neighboring cluster's selected incidences. Two cluster steps and their complete endpoint data fit within this radius, using the phase-3 descriptions. Generate each item's random choice once, indexed by its original edge identity, and cache it. Thus two endpoints always use the same choice.

5. *Residual cleanup*, radius $100(q+3)$: simulate the deterministic algorithm on R. When this phase first processes any member of a bad active cluster, query the corresponding residual vertex and all its private leaf vertices, in a fixed order, and cache their outputs. Later members reuse these outputs. A virtual step projects to at most 33 original hops, and a leaf stays at its owner's cluster. The radius-q virtual views for this block, including the needed incidence data and bad/good labels, are therefore within the stated original radius. The phase returns its cluster's complete selected-item directions, combining surviving marks with all residual outputs, including directions on private-leaf edges. At a good active cluster, the label instead records its marked witnesses and the fixed directions already determined there; directions controlled by a bad neighbor are read from that neighbor in the final phase.

6. *Final orientations*, radius 100: reconstruct the direction of every original boundary edge from the earlier labels at its selected endpoint or endpoints. Orient each active cluster's internal tree toward the tail of a selected outgoing item; absorbing clusters use their stored solutions. Return all incident directions of the queried vertex. If an active cluster has no outgoing item on an unsuccessful tape, orient its tree toward its least-identifier vertex instead. All members use the same complete data and tie-breaking rules, so their reports agree.

For clarity, a phase-5 label at a good cluster need not predict the output of a neighboring bad cluster. Phase 6 has the completed earlier-phase labels throughout its fixed-radius view, so it obtains that direction at the bad endpoint. Likewise, private leaf queries use the same global instance of the componentwise deterministic algorithm and give consistent directions at both ends of every virtual edge. All virtual vertices are eventually queried when their original clusters are processed.

Each phase is a valid online procedure with the prescribed radius on its earlier-phase-labeled input. By Lemma 5.1, the total original radius is at most $100q + 632$, hence is $O_c(\log\log\log n)$. This includes all preprocessing and all lifting overheads, even on unsuccessful tapes.

Fix now the original input graph and its complete external query order. The cone simulation fixes the query order of every preprocessing phase independently of all random bits. Consequently the completed centers, clusters, selected incidences, and quotient Q are fixed independently of the marking coins. This is exactly the hypothesis needed for (7)–(11). The virtual residual query order may depend on which clusters are bad, but that causes no difficulty: the deterministic cleanup works for every order whenever the component bound holds.

Proof of Theorem 6.1. The serialized execution of the six phases is correct on the shattering event: Lemma 6.2 protects good quotient vertices, the componentwise cleanup satisfies all bad ones, and the cluster lifting satisfies every original degree-three vertex. The online simulation produces exactly those outputs by Lemma 5.1. The shattering event has probability at least $1 - n^{-c}$, and the radius bound holds unconditionally, as just established. □

7 Completing the upper bounds

Proof of Theorem 1.2. For the deterministic result, apply Theorem 5.4 with $M = n$ to obtain a subcubic online algorithm of radius $O(\log \log n)$. Theorem 3.1 gives the same asymptotic dynamic radius. Lemma 2.1 extends it to general graphs, and Section 3.4 handles vertex insertions with incident edges.

For the randomized result, Theorem 6.1 supplies an online radius $O_c(\log \log \log n)$ and error n^{-c} on every fixed subcubic graph and fixed query order. Apply Theorem 4.1, then the same degree reduction and vertex-update refinement. For a fixed original insertion sequence all these reductions are deterministic. The one static success event in the transfer guarantees legal orientations at every dynamic time, with no additional probability loss. □

The transfer relies on both bounded-degree saturation and the freedom to leave low-degree vertices as sinks. It is therefore a statement about this particular orientation problem, rather than a general equivalence of the models. The proof also relies on insertions only: deletions could reopen a previously permanent neighborhood.

Neither upper bound is asserted to be optimal. For context, the lower bounds of Brandt and Göttlicher [9, Theorem 1.2 and Corollary 1.3], together with the dynamic-to-online simulation of [1], give a $2^{\Omega(\log^* n)}$ barrier for both deterministic dynamic locality and the oblivious randomized version considered here. Those lower bounds are background results, not ingredients in the self-contained upper-bound proofs. Determining the optimal locality within the remaining gaps is open.

References

[1] Amirreza Akbari, Navid Eslami, Henrik Lievonen, Darya Melnyk, Joona Särkijärvi, and Jukka Suomela. Locality in Online, Dynamic, Sequential, and Distributed Graph Algorithms. In *ICALP 2023*, LIPIcs 261, 10:1–10:20, 2023. doi:10.4230/LIPIcs.ICALP.2023.10. Full version: https://arxiv.org/abs/2109.06593v4.

[2] Alkida Balliu, Janne H. Korhonen, Fabian Kuhn, Henrik Lievonen, Dennis Olivetti, Shreyas Pai, Ami Paz, Joel Rybicki, Stefan Schmid, Jan Studený, Jukka Suomela, and Jara Uitto. Sinkless Orientation Made Simple. In *SOSA 2023*, pages 175–191, 2023. doi:10.1137/1.9781611977585.ch17. Full version: https://arxiv.org/abs/2108.02655v2.

[3] Mohsen Ghaffari, David G. Harris, and Fabian Kuhn. On Derandomizing Local Distributed Algorithms. In *FOCS 2018*, pages 662–673, 2018. doi:10.1109/FOCS.2018.00069. Full version, 17 September 2019: https://arxiv.org/abs/1711.02194v4. Theorem numbers in this note refer to that full version.

[4] Mohsen Ghaffari and Hsin-Hao Su. Distributed Degree Splitting, Edge Coloring, and Orientations. In *SODA 2017*, pages 2505–2523, 2017. doi:10.1137/1.9781611974782.166. Full version: https://arxiv.org/abs/1608.03220.

[5] Mohsen Ghaffari, Fabian Kuhn, and Yannic Maus. On the Complexity of Local Distributed Graph Problems. In *STOC 2017*, pages 784–797, 2017. doi:10.1145/3055399.3055471. Full version: https://arxiv.org/abs/1611.02663v2.

[6] Alkida Balliu, Juho Hirvonen, Darya Melnyk, Dennis Olivetti, Joel Rybicki, and Jukka Suomela. Local Mending. In *SIROCCO 2022*, LNCS 13298, pages 1–20, 2022. doi:10.1007/978-3-031-09993-9_1. Full version: https://arxiv.org/abs/2102.08703v4.

[7] Yann Bourreau, Filippo Casagrande, Tim Göttlicher, and Ananth Narayanan Karunattillam. Dynamic 3-Coloring of Bipartite Graphs with Optimal Locality. Preprint, 14 October 2026. https://arxiv.org/abs/2610.19710v1.

[8] Amirreza Akbari, Xavier Coiteux-Roy, Francesco d'Amore, François Le Gall, Henrik Lievonen, Darya Melnyk, Augusto Modanese, Shreyas Pai, Marc-Olivier Renou, Václav Rozhoň, and Jukka Suomela. Online Locality Meets Distributed Quantum Computing. Conference version in *STOC 2025*. doi:10.1145/3717823.3718211. We refer to the corrected full version of 13 February 2026: https://arxiv.org/abs/2403.01903v4.

[9] Sebastian Brandt and Tim Göttlicher. A Post-Quantum Lower Bound for the Distributed Lovász Local Lemma. In *SODA 2026*, 2026. doi:10.1137/1.9781611978971.119. Full version: https://arxiv.org/abs/2510.15698v1.

[10] Alexandre Nolin. Brief Announcement: Sinkless Orientation Made Trivial. In *PODC 2026*, pages 333–336, 2026. doi:10.1145/3796701.3815953. Author version: https://hal.science/hal-05678283v1.